Reinforcement Learning for Congestion Control: A Review of the 50 Most-Cited Studies

Gaurab Baral Nityashree Vangala

CS 6043: Computer Networking, Fall 2026

1 Introduction

Every time we load a web page or join a video call, a congestion control (CC) algorithm on the sending machine decides how fast to send. If it sends too slowly, the link sits idle. If it sends too fast, packets pile up in router buffers, delay grows, and eventually packets are dropped. The first widely used answer came from Jacobson after the Internet's congestion collapses of 1986: start slowly, grow the sending window a little each round trip, and cut it in half when a packet is lost [1]. Versions of that idea still run on most computers today.

The networks those rules were written for have changed a lot. Data now crosses gigabit fiber, crowded Wi-Fi, cellular links whose capacity jumps around every few milliseconds, satellite paths, and data-center fabrics where a round trip takes tens of microseconds. Researchers have answered with a long line of hand-designed algorithms: CUBIC [2] and BBR [3] for the Internet, DCTCP [4] for data centers, and many more. Each encodes a designer's guess about the network and works best in the setting it was tuned for.

Reinforcement learning (RL) offers another way to build a controller. Instead of writing the rules by hand, we let an agent try sending rates, watch what happens, and learn from a reward that scores throughput, delay and loss. Remy [5] and PCC [6] showed that machine-generated or online-learned controllers could beat hand-tuned ones, and Aurora [7] turned Internet congestion control into a deep RL problem in 2019. After that the number of papers grew quickly. We picked this topic because the results we kept running into looked very good on paper, yet almost nobody seemed to run these controllers in practice, and we wanted to know why.

We organized the review around three research questions:

RQ1 **Design.** What do RL congestion controllers observe, what do they change, and what reward do they learn from?

RQ2 **Testing.** How are they trained and tested, and do they still work on networks they did not see during training?

RQ3 **Deployment.** What stops operators from using them, and what research could fix that?

Our search followed the PRISMA 2020 guideline for systematic reviews [8] and found 362 eligible studies published between 2016 and 2026. Reading all of them in depth was not possible in one semester, so this report reviews the 50 with the most citations. We also ran a small experiment of our own: we trained two PPO controllers on different sets of simulated links and tested both on conditions they had not seen, to check one of the patterns we found in the literature.

2 Detailed Description

2.1 How congestion control works

A good congestion controller has several jobs at once. It should fill the bottleneck link, avoid building long queues or losing packets, share the link fairly with other flows, react quickly when conditions change, and get along with the algorithms already running on the Internet. Ware et al. argue that a new algorithm should be judged by how much *harm* it does to existing traffic [9], since a scheme that starves CUBIC flows will never be allowed onto a shared network.

A sender cannot see inside the network, so it has to guess what is happening from the signals it gets back: packet loss, the round-trip time (RTT) and how it is trending, the rate at which data is being delivered, and Explicit Congestion Notification (ECN) marks that routers can set instead of dropping packets. It acts through one of two knobs. The *congestion window* (cwnd) limits how much unacknowledged data can be in flight, and the *pacing rate* spaces packets out in time.

Classic algorithms differ in which signal they trust. Reno grows the window by one packet per RTT and halves it on loss [10]. CUBIC, the Linux default, grows the window as a cubic function of the time since the last loss, which lets it recover faster on long, fast paths [2]. Vegas backs off as soon as the RTT rises above its minimum [11], which keeps queues short but loses out to loss-based flows. BBR builds a model instead: it estimates the bottleneck bandwidth and the minimum RTT and paces at the estimated bandwidth [3]. Routers can help too. Active queue management schemes such as RED [12] and CoDel [13] drop or mark packets before the buffer fills, and in data centers DCTCP cuts its window in proportion to the fraction of ECN-marked packets [4]. DCTCP's marking threshold is usually tuned by hand, which is one place RL has been tried.

2.2 Reinforcement learning in brief

RL treats a problem as a loop between an agent and an environment, written as a Markov decision process. At each step t the agent observes a state s_t, picks an action a_t using its policy π, receives a reward r_t, and the environment moves to a new state s_{t+1}. The agent's goal is a policy that maximizes the expected discounted return $\mathbb{E}[\sum_t \gamma^t r_t]$, where $\gamma < 1$ makes near-term rewards count more than distant ones [14]. A sender only sees its own feedback and never the full state of the network, so congestion control is really a partially observable problem. Most designs deal with this by giving the agent a short history of recent measurements.

The studies we reviewed use several families of RL methods. *Value-based* methods learn how good each action is in each state; Q-learning does this with a table and the Deep Q-Network (DQN) with a neural network [15]. *Policy-gradient* methods such as PPO [16] adjust the policy directly and handle continuous actions like "multiply the rate by 1.03" easily. *Off-policy actor-critic* methods such as TD3 [17] reuse past experience stored in a replay buffer, which makes them more sample efficient. A few studies use *offline RL*, which learns only from logged data without trying new actions online, *imitation learning*, which copies an expert, *contextual bandits*, which treat every decision as a one-step problem, or *multi-agent RL*, where several agents such as competing flows learn together.

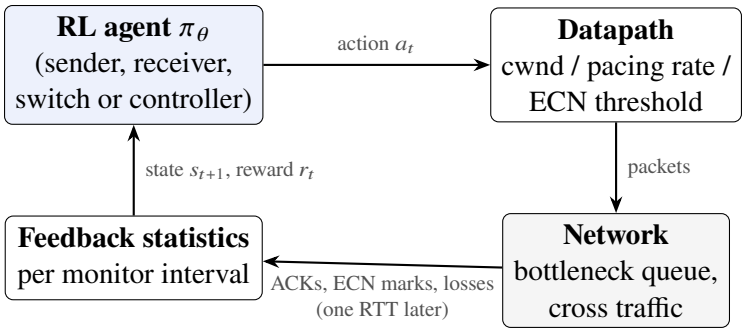


Figure 1: The usual RL congestion-control loop. The agent acts once per monitor interval and sees the results about one RTT later.

2.3 Congestion control as an RL problem

Figure 1 shows the formulation that Aurora made popular [7], and most later papers follow it closely. Time is split into short *monitor intervals*. At the start of each interval the agent chooses an action. When the acknowledgments for that interval come back, the sender computes statistics such as throughput, average RTT and loss rate, turns them into the next state, and computes a reward.

Within this loop, designers have to make five choices.

1. **Where the agent runs.** Most agents run at the sender. For real-time video the agent often runs at the receiver, and in data centers it can sit in each switch or in a central controller.
2. **What it observes.** Typical inputs are recent throughput, RTT, the RTT gradient, loss rate and the current window or rate. Aurora deliberately used ratios (latency gradient, latency ratio and sending ratio) so that the same policy could work at different link speeds.
3. **What it changes.** The action can set the window or rate directly, tune a parameter of a classic algorithm, choose among several existing algorithms, or set something in the network such as an ECN threshold.
4. **What it is rewarded for.** Most rewards add up throughput and subtract penalties for delay and loss.
5. **How often it acts.** Running a neural network on every ACK is too slow at high speeds, so most designs act once per interval.

The reward deserves a closer look, because it decides what the controller will do. Aurora's reward is $10 \cdot \text{throughput} - 1000 \cdot \text{latency} - 2000 \cdot \text{loss}$, with throughput in packets per second, latency in seconds and loss as a fraction. To see what the weights mean, take a 10 Mbit/s link (about 830 packets per second of 1500 bytes) with a 50 ms RTT. A sender that fills the link earns about 8,300 from throughput and loses 50 for the base latency. If it overshoots and builds a 100 ms queue, it loses another 100. That penalty is small next to the throughput term, so this reward will tolerate quite a bit of queueing to keep the link full. A reward with a larger delay weight would produce a more cautious controller. In other words, the weights act as the specification of the controller, and two papers with different weights are really solving different problems.

The problem has two more difficulties. An action's result arrives one RTT later, mixed with the effects of every other flow, so the agent struggles to tell which action caused what. And the controller will run on networks unlike its training ones, so behavior under that shift is the main test

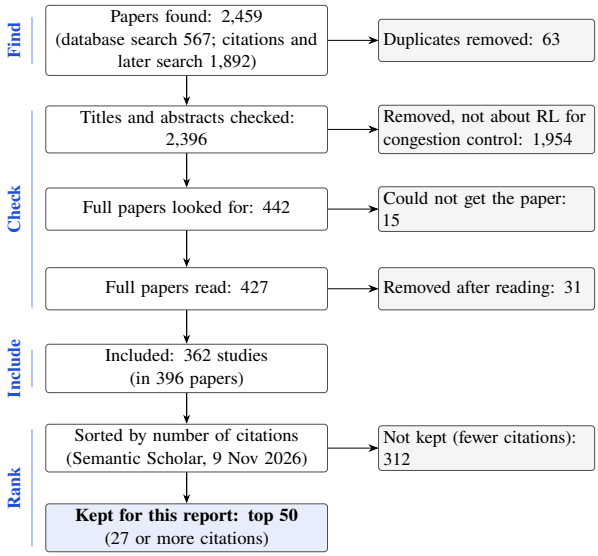


Figure 2: How we chose the 50 studies, drawn in the style of a PRISMA 2020 flow diagram [8]. One study can appear in several papers, such as a conference paper and its journal version.

of its quality.

2.4 How we chose the papers

We searched OpenAlex, arXiv and the USENIX NSDI and ATC proceedings for 2016 to 2026 with the query `("reinforcement learning" OR "imitation learning") AND ("congestion control" OR "congestion window" OR "ECN")`. Our proposal also listed IEEE Xplore, the ACM Digital Library and Scopus, but their programmatic interfaces need institutional keys, so we used OpenAlex, which indexes most of the same venues. We added forward citation searching (papers that cite well-known studies such as Aurora and Orca), backward citation searching (the reference lists of recent studies), and a later search with wider terms such as “Q-learning”, “bandit” and “AQM”. These steps follow the advice of Kitchenham and Charters for systematic reviews in computing [18].

We kept a study if an RL or imitation-learning method controls congestion in a packet network, or if the study builds a framework to train or test such controllers. We removed RL for other tasks such as routing, methods with no learning component, and other surveys. Figure 2 shows how many records survived each step. To make the review manageable we then ranked the 362 included studies by their citation count in Semantic Scholar on 9 November 2026 and kept the top 50; the fiftieth study has 27 citations.

For each of the 50 studies we recorded the network setting, where the agent runs, the RL method, the state, action and reward, and the highest level of testing it reached: *simulation*, *emulation* (real network software over software-imposed links), or a *real network*. We also noted whether the study tested conditions outside its training range, whether it measured fairness, and whether it released code. We call a design *hybrid* when a classic algorithm or safety rule stays in the control loop after deployment. Finally, each design study gets a practice score from 0 to 5, one point each for reporting the test environment, using emulation or a real network, testing generalization, measuring fairness, and releasing code.

Table 1: Landmark designs among the 50. Level is the highest testing environment the study reached (Emu = emulation).

Study	Action	Learning signal	Algorithm	Level
Indigo [19]	cwnd step every 10 ms	imitate an oracle that knows the emulated link's ideal cwnd	imitation, LSTM	Real
Aurora [7]	rate multiplier per interval	weighted throughput, latency, loss	PPO	Emu
Orca [20]	scale CUBIC's cwnd by 2^{2a}	throughput minus loss, scaled by delay	TD3	Real
MOCC [21]	rate	preference-weighted throughput, delay, loss	PPO	Real
Genet [22]	(training method)	Aurora's reward with a curriculum	PPO	Real
Sage [23]	cwnd	logged behavior of existing heuristics	offline RL	Real
Astraea [24]	rate	throughput, delay, loss, fairness	multi-agent	Real
Mutant [25]	select a kernel CC scheme	power (throughput / delay)	LinUCB bandit	Real

2.5 The designs among the 50

Of the 50 studies, 36 target the wide-area Internet or a generic wired bottleneck, 6 target wireless or cellular networks, 4 target IoT or sensor networks, 3 target data centers and 1 targets satellite links. Of the 43 studies that propose a controller, 33 run the agent at the sender. Table 1 lists the designs that later papers build on most often, and the paragraphs below group the rest.

Early Q-learning controllers. The oldest designs in the set use Q-learning over small state spaces that the authors split into buckets by hand. QTCP approximates the Q-function with Kanerva coding and reports 59.5% higher throughput than rule-based TCP [26]. These designs are simple to understand, but hand-made buckets do not scale to the wide range of link speeds and delays seen on the Internet.

Clean-slate deep RL. Aurora [7] set the template most later work copies. It was trained with PPO on simulated links of 1.2 to 6 Mbit/s and kept near-full utilization in emulation from 1 to 128 Mbit/s, a range far wider than its training data. It did poorly at 1 ms latency, though, because about 1 ms of packet-processing noise that never appeared in training swamped its latency signal. We return to this failure in our own experiment. Indigo, released with the Pantheon testbed [19], took a different route and imitated an oracle that knew the best window for each emulated link. MVFST-RL [27] pointed out a problem that simulators hide: in a real network stack, packets keep flowing while the agent is still computing. It runs the policy asynchronously inside Facebook's QUIC implementation.

Hybrids built on classic control. 15 of the 43 design studies (35%) are hybrids. The common idea is to let a classic algorithm react to every ACK while the agent adjusts it more slowly. In Orca [20], CUBIC runs in the Linux kernel, and once per monitoring period a TD3 agent multiplies CUBIC's window by 2^{2a} with $a \in [-1, 1]$. Orca performed more consistently than clean-slate learned controllers and needed less computation. Spine [28] has a slow agent that hands sub-policies to a light executor, which cuts control overhead by at least 80%. Other designs learn *which* algorithm to run. TCP-RL [29] chooses the initial window and picks among 14 TCP variants on the web servers of a large search engine, and Mutant [25] switches between kernel protocols using a contextual bandit.

Learning from logs. Exploring online on real users' traffic is risky, so some studies learn from data that already exists. Sage [23] trains offline on traces of existing heuristics and ends up outperforming the heuristics it learned from. Concerto [30] imitated over one million logged video-call sessions.

Several objectives and fairness. A fixed reward fixes one trade-off between throughput and delay. MOCC [21] instead feeds the policy a preference vector, so one model can serve different applications, and it adapts to a new objective 14.2 times faster than earlier work. Fairness is a separate problem: a policy trained alone tends to be greedy when copies of it share a link. Astraea [24] trains several flows together with a fairness term in the reward and converges to a fair share up to 8.4 times faster.

Other settings. For real-time video, the controller is a bandwidth estimator that sets the encoder's bitrate. Loki [31] combines an RL model with a rule-based one, and OnRL [32] learns online from mobile users, with a rule-based fallback that takes over when the learned policy misbehaves. In data centers, ACC [33] runs an agent in every switch to tune ECN thresholds, and RL-CC [34] treats RDMA congestion control as a multi-agent problem. All IoT studies whose testing level we could find stop at simulation.

2.6 How the studies are tested

We sort testing environments by how much of the real system they include. *Simulators* such as ns-3 model both the network and the hosts, and they leave out the operating system's timers, pacing and ACK batching. *Emulators* such as Mininet [35] and Mahimahi [36] run real network software over links shaped in software. *Testbeds and real networks*, such as Pantheon's machines spread around the world [19], cloud paths, and production traffic, put the controller on physical links. Four of the 50 studies are testing frameworks (Pantheon, MVFST-RL, OpenNetLab and Iroko), and two verify trained policies formally; whiRL proves some properties of Aurora and finds counterexamples to others [37].

3 Results

3.1 How well the studies are tested

Table 2 compares the 50 most-cited studies with the other 312 eligible studies. Of the 48 top studies with an experimental evaluation, 18 reach a real network, 11 stop at emulation, 13 stop at simulation and 6 do not say.

Observation 1: the most-cited studies test on real networks more often, but most still do not. The top 50 reach a real network more than twice as often as the rest (43% against 19%) and rely on simulation alone about half as often. Part of this gap comes from how much we could read: we had the full text of the top studies about twice as often, and abstracts often skip testing details.

Observation 2: many citations do not mean strong evidence. The median top design scores only 2 out of 5 on our practice score. Just 6 score four or more: Orca scores 5, and Aurora, Genet, MOCC, Astraea and Mutant score 4. We expected the most influential papers to set a higher bar than this.

Table 2: The 50 most-cited studies against the other 312 eligible studies. Generalization, fairness, code and hybrid are counted over design studies. Testing shares are over studies whose testing level is known.

	Top 50	Other 312
Design studies	43	282
Reach a real network	18/42 (43%)	39/204 (19%)
Simulation only	13/42 (31%)	131/204 (64%)
Test outside training range	7 (16%)	28 (10%)
Measure fairness	12 (28%)	51 (18%)
Release code	7 (16%)	18 (6%)
Hybrid (classic part in the loop)	15 (35%)	78 (28%)
First published 2023 or later	4 (8%)	218 (70%)
Median practice score (0–5)	2	1

Observation 3: few studies test outside their training range. Only 7 of the 43 designs test on conditions they were not trained on, and only 7 release code. Without public code, later papers keep comparing against the same few open-source controllers, mostly Aurora and Orca. Deep RL results also change with random seeds [38], so a result from one run is hard to trust. Where generalization does work, it comes from training on a broad, deliberately chosen set of environments or from training on the deployment's own data. Genet [22], for example, picks training environments where the RL policy is doing worse than rule-based baselines, and its policies beat normally trained ones on real paths.

Observation 4: independent tests are much less positive. Recent independent evaluations are not in our top 50, probably because they are too new to have many citations, but they matter for **RQ2**. Giacomoni and Parisis found that public RL controllers grab bandwidth quickly and handle random loss, but cause extra loss, are unfair in almost every setup, and react slowly when conditions change [39]. Mazilu et al. report similar problems against CUBIC and BBRv3 [40]. Ghermezian et al. ran an Aurora-style controller on a real 10 Gbit/s wide-area path, where it reached 0.5% average utilization while CUBIC and BBR reached up to 9.9 and 9.7 Gbit/s [41]. Someone who read only the most-cited papers would not see any of this.

3.2 Our experiment: what the training data does to a controller

Observation 3 raised a question the papers could not answer: with everything else fixed, how much does the choice of training links change a controller's behavior on unseen links? We built a small simulator to find out.

Setup. The simulator models one flow through one drop-tail bottleneck queue and uses Aurora's formulation. The agent acts every 20 ms, its feedback arrives one base RTT later, and it observes the latency gradient, latency ratio and sending ratio for the last ten intervals. Each action changes the rate by at most 5%, and the reward is utilization minus queueing delay (in RTTs) minus three times the loss rate. We trained PPO [16] for 5 million steps with 4 random seeds on each of two sets of links. The *narrow* set used 1 to 5 Mbit/s, 40 to 60 ms RTT, buffers of 1 to 2 bandwidth-delay products (BDP), and no loss or jitter, which is close to Aurora's training range. The *broad* set used 1 to 100 Mbit/s, 10 to 200 ms, 0.2 to 5 BDP, up to 3% random loss, up to 10 ms of RTT jitter, and

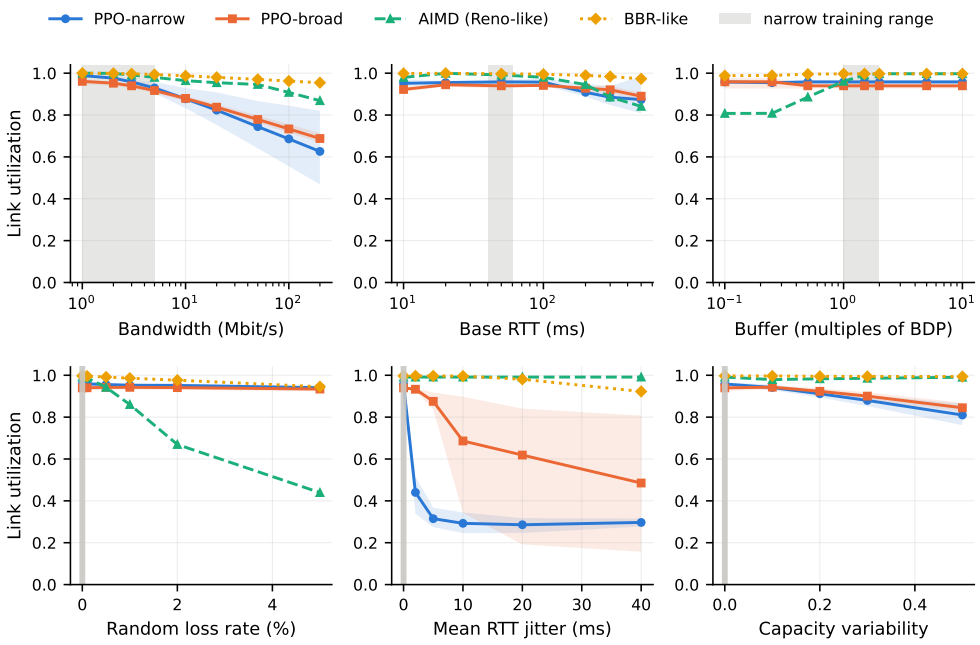


Figure 3: Our experiment: link utilization when one factor is changed around the reference link. Lines are means over the training seeds that converged, and shaded bands show their spread. The gray band marks the narrow training range.

Table 3: Summary of our experiment, averaged over converged seeds. "Queue" is the mean queueing delay in multiples of the base RTT. One of the 4 broad seeds failed to learn and is left out; all narrow seeds converged.

	In distribution		Utilization (%) under one shifted factor			
Controller	Util. (%)	Queue (RTT)	200 Mbit/s	5% loss	10 ms jitter	Worst case
PPO-narrow	96	0.03	63	94	29	29 (20 ms jitter)
PPO-broad	94	0.09	69	93	69	49 (40 ms jitter)
AIMD (Reno-like)	99	0.72	87	44	99	44 (5% loss)
BBR-like	100	0.09	95	95	100	92 (40 ms jitter)

capacity that changes over time. We compared both against a Reno-like AIMD controller and a BBR-like controller, changing one factor at a time around a reference link of 3 Mbit/s, 50 ms and 1.5 BDP. Each training run took about 9 minutes on one CPU core.

What we found. Figure 3 and Table 3 show the results.

- **On familiar links the learned controllers do well.** Both PPO policies reach at least 94% utilization with queues of only 0.03 to 0.09 RTT, while AIMD keeps a queue of 0.72 RTT.
- **A little delay noise breaks the narrow policy.** With 10 ms of RTT jitter, which is normal on Wi-Fi and cellular links, the narrow policy's utilization drops from 96% to 29%. Its latency inputs read the noise as a growing queue, so it keeps slowing down. AIMD ignores delay and BBR-like filters it, so both stay near full utilization. This is the same failure the Aurora authors saw at 1 ms latency.

Table 4: Studies among the 50 that report production use or tests with real users.

System	Setting	Learned component	Classic in loop	Reported scale
TCP-RL [29]	Web search	initial window; algorithm selection	yes	search engine, over a year; 23% shorter short flows
Concerto [30]	Video calls	bitrate policy imitated from logs	no	production video-telephony app
OnRL [32]	Mobile video	per-user online RL, combined model	yes	543 h, 151 users; 14.2% fewer stalls
Loki [31]	Real-time video	RL fused with a rule-based model	yes	101 million sessions; 26 to 44% fewer stalls
ACC [33]	Data center	per-switch ECN threshold	yes	300 machines; 30% lower flow completion time

- **Broader training helps only inside its range.** The broad policy, which saw jitter up to 10 ms, keeps 69% utilization at 10 ms but falls to 49% at 40 ms. It also keeps a longer queue on the reference link, and one of its four seeds never learned at all and stayed at 3% utilization.
- **The learned controllers fail in different places than classic ones.** With 5% random loss the PPO policies keep 93 to 94% utilization while AIMD falls to 44%. A test suite built around the known weaknesses of classic algorithms would make the learned controllers look great and miss the jitter problem entirely.
- **Fast links expose a design limit.** At 200 Mbit/s the PPO policies reach only 63 to 69% utilization, compared with 95% for BBR-like. Because each action can change the rate by at most 5%, climbing two orders of magnitude takes seconds no matter what the training data looked like.

Our simulator is simple: it has one flow, a fluid queue, no competing traffic and small policies, so we do not claim these numbers for any published controller. Still, it showed that a learned controller's weak spots follow from its inputs, and a wider training set moves them instead of removing them.

3.3 Deployment

Table 4 lists the 5 studies among the 50 that report use in production or tests with real users.

Observation 5: the deployed systems share a pattern. Every one of them is run by a company that controls the senders or the switches, so it can monitor the controller, test it on part of its traffic and roll it back. Each one optimizes a single service metric such as transfer time, video stalls or storage throughput. Four of the five keep a classic algorithm or safety rule in the loop, and the fifth, Concerto, learned only from logged sessions and never explored on live traffic. None of them is a general-purpose controller for Internet traffic.

What holds deployment back. Putting the papers and our experiment together, we see five barriers. The first is *generalization*: our experiment and the independent tests both show learned controllers failing on ordinary conditions they were not trained for, and deployed systems avoid this by training on their own traffic or limiting the damage a bad decision can do. The second is *safety*: formal verification can check a trained policy after the fact [37], but deployed systems rely on runtime

fallbacks like OnRL's. The third is *fairness*: only 12 of 43 designs measure it, and almost none test against CUBIC and BBR, the traffic a new controller would actually share links with. The fourth is *overhead*: Orca and Spine run the agent on a slower timescale, and NVIDIA's follow-up to RL-CC turned its policy into decision trees, cutting inference time 500 times so it fit on a network card [42]. The last is *maintenance*: the papers rarely discuss when to retrain, how to notice that traffic has changed, or how to roll back a bad model.

LLM-designed controllers. None of the 50 uses a large language model, since this work only started around 2024, but our wider search found a few. PolicySmith uses an LLM to write congestion-control heuristics that run in the Linux kernel [43], and Trailblazer calls a small LLM only in difficult network conditions and beat a tuned industrial policy in a large A/B test [44]. Both keep the LLM off the per-packet path, like Orca's split between fast and slow decisions. LLM-written code can be reviewed like any heuristic, but the generator can overfit to the emulator that scores it.

Limitations of our review. Citation counts favor older papers and well-known venues: only 4 of the 50 first appeared in 2023 or later, against 70% of the other studies. We could read the full text of only 19 of the 50, so practices that abstracts tend to skip are probably undercounted. Screening and coding were done in a single pass with tool support; we are now each checking the 50 studies independently to measure how often we agree.

4 Summary

We reviewed the 50 most-cited of 362 eligible studies on RL for congestion control and ran a small experiment of our own.

- **Design (RQ1).** Most controllers watch recent throughput, delay and loss, change the window or rate, and learn from a weighted reward whose weights set their behavior. 15 of 43 designs keep a classic algorithm or safety rule in the loop.
- **Testing (RQ2).** 43% of the top studies with a known testing level reach a real network, compared with 19% of the others. Only 16% of designs test outside their training range and 16% release code. Our experiment shows them failing on ordinary delay noise.
- **Deployment (RQ3).** All 5 deployed systems are run by companies that own the endpoints or switches and optimize one narrow metric, and four of the five keep a classic component in the loop.

This project changed how we read RL papers: a high utilization number on a simulated link says little until we know what the controller was trained and tested on. Next, we would test controllers against CUBIC and BBR on shared links [9] and extend the review to the work published since 2023.

Contributions of Group Members

Gaurab Baral and Nityashree Vangala contributed equally to this report and to the class presentation.

References

[1] V. Jacobson, "Congestion avoidance and control," in *Symposium proceedings on Communications architectures and protocols*. ACM, Aug. 1988, pp. 314–329.

[2] S. Ha, I. Rhee, and L. Xu, "CUBIC: a new TCP-friendly high-speed TCP variant," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 64–74, 2008.

[3] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-Based Congestion Control: Measuring bottleneck bandwidth and round-trip propagation time," *Queue*, vol. 14, no. 5, pp. 20–53, Oct. 2016.

[4] M. Alizadeh, A. Greenberg, D. A. Maltz *et al.*, "Data center TCP (DCTCP)," in *Proceedings of the ACM SIGCOMM 2010 conference*. ACM, Aug. 2010, pp. 63–74.

[5] K. Winstein and H. Balakrishnan, "TCP ex machina: computer-generated congestion control," in *Proceedings of the ACM SIGCOMM 2013 conference on SIGCOMM*. ACM, Aug. 2013, pp. 123–134.

[6] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "PCC: Re-architecting Congestion Control for Consistent High Performance," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2015, pp. 395–408.

[7] N. Jay, N. H. Rotman, P. B. Godfrey, M. Schapira, and A. Tamar, "A Deep Reinforcement Learning Perspective on Internet Congestion Control," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, vol. 97, 2019, pp. 3050–3059.

[8] M. J. Page, J. E. McKenzie, P. M. Bossuyt *et al.*, "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, p. n71, Mar. 2021.

[9] R. Ware, M. K. Mukerjee, S. Seshan, and J. Sherry, "Beyond Jain's Fairness Index: Setting the Bar For The Deployment of Congestion Control Algorithms," in *Proceedings of the 18th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2019, pp. 17–24.

[10] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," RFC 5681, IETF, 2009.

[11] L. Brakmo and L. Peterson, "TCP Vegas: end to end congestion avoidance on a global Internet," *IEEE Journal on Selected Areas in Communications*, vol. 13, no. 8, pp. 1465–1480, 1995.

[12] S. Floyd and V. Jacobson, "Random early detection gateways for congestion avoidance," *IEEE/ACM Transactions on Networking*, vol. 1, no. 4, pp. 397–413, 1993.

[13] K. Nichols and V. Jacobson, "Controlling queue delay," *Communications of the ACM*, vol. 55, no. 7, pp. 42–50, 2012.

[14] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[15] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.

[16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[17] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, vol. 80, 2018, pp. 1587–1596.

[18] B. Kitchenham and S. Charters, "Guidelines for Performing Systematic Literature Reviews in Software Engineering," Keele University and Durham University, Tech. Rep. EBSE-2007-01, 2007.

[19] F. Y. Yan, J. Ma, G. D. Hill *et al.*, "Pantheon: The Training Ground for Internet Congestion-Control Research," in *2018 USENIX Annual Technical Conference (USENIX ATC)*, 2018.

[20] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Classic Meets Modern: a Pragmatic Learning-Based Congestion Control for the Internet," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. ACM, 2020, pp. 632–647.

[21] Y. Ma, H. Tian, X. Liao *et al.*, "Multi-objective congestion control," in *Proceedings of the Seventeenth European Conference on Computer Systems*. ACM, Mar. 2022, pp. 218–235.

[22] Z. Xia, Y. Zhou, F. Y. Yan, and J. Jiang, "Genet: automatic curriculum generation for learning adaptation in networking," in *Proceedings of the ACM SIGCOMM 2022 Conference*. ACM, Aug. 2022, pp. 397–413.

[23] C.-Y. Yen, S. Abbasloo, and H. J. Chao, "Computers Can Learn from the Heuristic Designs and Master Internet Congestion Control," in *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM, 2023, pp. 255–274.

[24] X. Liao, H. Tian, C. Zeng, X. Wan, and K. Chen, "Astraea: Towards Fair and Efficient Learning-based Congestion Control," in *Proceedings of the Nineteenth European Conference on Computer Systems*. ACM, Apr. 2024, pp. 99–114.

[25] L. Pappone, A. Sacco, and F. Esposito, "Mutant: Learning Congestion Control from Existing Protocols via Online Reinforcement Learning," in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2025.

[26] W. Li, F. Zhou, K. R. Chowdhury, and W. Meleis, "QTCP: Adaptive Congestion Control with Reinforcement Learning," *IEEE Transactions on Network Science and Engineering*, vol. 6, no. 3, pp. 445–458, 2019.

[27] V. Sivakumar, O. Delalleau, T. Rocktäschel *et al.*, "MVFST-RL: An Asynchronous RL Framework for Congestion Control with Delayed Actions," arXiv:1910.04054, 2019.

[28] H. Tian, X. Liao, C. Zeng, D. Sun, J. Zhang, and K. Chen, "Efficient DRL-Based Congestion Control With Ultra-Low Overhead," *IEEE/ACM Transactions on Networking*, vol. 32, no. 3, pp. 1888–1903, 2024.

[29] X. Nie, Y. Zhao, Z. Li *et al.*, "Dynamic TCP Initial Windows and Congestion Control Schemes Through Reinforcement Learning," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1231–1247, 2019.

[30] A. Zhou, H. Zhang, G. Su *et al.*, "Learning to Coordinate Video Codec with Transport Protocol for Mobile Video Telephony," in *The 25th Annual International Conference on Mobile Computing and Networking*. ACM, Oct. 2019, pp. 1–16.

[31] H. Zhang, A. Zhou, Y. Hu *et al.*, "Loki: improving long tail performance of learning-based real-time video adaptation by fusing rule-based models," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*. ACM, Oct. 2021, pp. 775–788.

[32] H. Zhang, A. Zhou, J. Lu *et al.*, "OnRL: improving mobile video telephony via online reinforcement learning," in *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. ACM, 2020, pp. 1–14.

[33] S. Yan, X. Wang, X. Zheng, Y. Xia, D. Liu, and W. Deng, "ACC: automatic ECN tuning for high-speed datacenter networks," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. ACM, Aug. 2021, pp. 384–397.

[34] C. Tessler, Y. Shpigelman, G. Dalal *et al.*, "Reinforcement Learning for Datacenter Congestion Control," *ACM SIGMETRICS Performance Evaluation Review*, vol. 49, no. 2, pp. 43–46, Jan. 2022.

[35] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*. ACM, Oct. 2010, pp. 1–6.

[36] R. Netravali, A. Sivaraman, S. Das *et al.*, "Mahimahi: Accurate Record-and-Replay for HTTP," in *2015 USENIX Annual Technical Conference (USENIX ATC)*, 2015, pp. 417–429.

[37] T. Eliyahu, Y. Kazak, G. Katz, and M. Schapira, "Verifying learning-augmented systems," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. ACM, Aug. 2021, pp. 305–318.

[38] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep Reinforcement Learning That Matters," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018.

[39] L. Giacomoni and G. Parisis, "Reinforcement Learning-based Congestion Control: A Systematic Evaluation of Fairness, Efficiency and Responsiveness," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*. IEEE, May 2024, pp. 1451–1460.

[40] M. Mazilu, L. Giacomoni, and G. Parisis, "Learning-Based vs Human-Derived Congestion Control: An In-Depth Experimental Study," arXiv, 2025.

[41] A. Ghermezian, K. Karpov, D. Kachan, V. Kirova, and E. Siemens, "PPO-Driven Reinforcement Learning Congestion Control Under High- BDP Wide-Area Deployment: A Scaling Analysis," 2026.

[42] B. Fuhrer, Y. Shpigelman, C. Tessler *et al.*, "Implementing Reinforcement Learning Datacenter Congestion Control in NVIDIA NICs," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, May 2023, pp. 331–343.

[43] R. Dwivedula, D. Saxena, A. Akella, S. Chaudhuri, and D. Kim, "Man-Made Heuristics Are Dead. Long Live Code Generators!" in *Proceedings of the 24th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2025, pp. 51–60.

[44] D. Wu, L. Kang, Z. Wang *et al.*, "Grounding Large Language Models as Generalizable Policies in Network Control," arXiv:2512.11839, 2025.